섹션 A – 연결 리스트 [이 섹션에서 1문항 선택]

temp = ll->head;
ListNode *temp;

안내: 문제용 프로그램 템플릿은 APAS 시스템에서 제공됩니다. 반드시
해당 템플릿을 사용하여 함수를 구현하세요.

1. (insertSortedLL) 사용자에게 정수를 입력받아

이를 오름차순 연결 리스트에 삽입하는 C 함수 insertSortedLL()을 작성하시오.
현재 연결 리스트에 이미 존재하는 정수는 삽입할 수 없다. 함수는
새 항목이 추가된 인덱스 위치를 반환해야 하며, 성공적으로 수행되지 못하면
-1을 반환해야 한다. 연결 리스트는 정렬된 리스트이거나
빈 리스트라고 가정해도 된다.

함수 원형은 다음과 같다:

```
int insertSortedLL(LinkedList *ll, int item);
```

현재 연결 리스트가 2,3,5,7,9라면,
insertSortedLL()를 값 8로 호출했을 때 결과 연결 리스트는 다음과 같다:
2,3,5,7,8,9.

함수는 새 항목이 추가된 인덱스 위치를 다음과 같이 반환해야 한다:

```
값 8은 인덱스 4에 추가되었다
```

현재 연결 리스트가 5,7,9,11,15라면,
insertSortedLL()를 값 7로 호출했을 때 결과 연결 리스트는 다음과 같다:
5,7,9,11,15.

이 경우 함수는 성공적으로 수행되지 못하므로(값 7을 연결 리스트에 삽입하지 못하므로)
-1을 반환해야 한다:

```
값 7은 인덱스 -1에 추가되었다
```

예시 입력과 출력은 다음과 같다:

```
1: 정수를 정렬된 연결 리스트에 삽입:
2: 가장 최근 입력값의 인덱스 출력: 3: 정렬된
연결 리스트 출력:
0: 종료:

메뉴를 선택하세요(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하세요: 2
결과 연결 리스트는: 2

메뉴를 선택하세요(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하세요: 3
결과 연결 리스트는: 2 3

메뉴를 선택하세요(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하세요: 5
```

```
결과 연결 리스트는: 2 3 5

메뉴를 선택하세요(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하세요: 7
결과 연결 리스트는: 2 3 5 7

메뉴를 선택하세요(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하세요: 9
결과 연결 리스트는: 2 3 5 7 9

메뉴를 선택하세요(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하세요: 8
결과 연결 리스트는: 2 3 5 7 8 9

메뉴를 선택하세요(1/2/3/0): 2
값 8은 인덱스 4에 추가되었다

메뉴를 선택하세요(1/2/3/0): 3
결과 정렬 연결 리스트는: 2 3 5 7 8 9

메뉴를 선택하세요(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하세요: 5
결과 연결 리스트는: 2 3 5 7 8 9

메뉴를 선택하세요(1/2/3/0): 2
값 5은 인덱스 -1에 추가되었다

메뉴를 선택하세요(1/2/3/0): 3
결과 정렬 연결 리스트는: 2 3 5 7 8 9

메뉴를 선택하세요(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하세요: 11
결과 연결 리스트는: 2 3 5 7 8 9 11

메뉴를 선택하세요(1/2/3/0): 2
값 11은 인덱스 6에 추가되었다

메뉴를 선택하세요(1/2/3/0): 3
결과 정렬 연결 리스트는: 2 3 5 7 8 9 11
```

2. (alternateMergeLL) 두 번째 리스트의 노드들을

첫 번째 리스트의 번갈아 가는 위치에 삽입하는 C 함수 alternateMergeLL()을 작성하시오.
두 번째 리스트의 노드는 첫 번째 리스트에 번갈아 삽입할 위치가 있을 때만 삽입해야 한다.

함수 원형은 다음과 같다:

```
void alternateMergeLL(LinkedList *ll1, LinkedList *ll2);
```

예를 들어 주어진 두 연결 리스트가 LinkedList1과 LinkedList2라면:
LinkedList1: 1,2,3
LinkedList2: 4,5,6,7

결과 연결 리스트는 다음과 같다:
LinkedList1: 1,4,2,5,3,6
LinkedList2: 7

첫 번째 리스트가 더 길면 두 번째 리스트는 빈 리스트가 되어야 한다. 예를 들어
주어진 두 연결 리스트가 LinkedList1과 LinkedList2라면:
LinkedList1: 1,5,7,3,9,11
LinkedList2: 6,10,2,4

결과 연결 리스트는 다음과 같다:
LinkedList1: 1,6,5,10,7,2,3,4,9,11
LinkedList2: 빈 리스트

예시 입력/출력 세션은 다음과 같다. 현재 연결 리스트가 Linked list
1: 1,2,3 이고 Linked list 2: 4,5,6,7 인 경우이다:

```
연결 리스트 1: 1 2 3
연결 리스트 2: 4 5 6 7

메뉴를 선택하세요(1/2/3/0): 3
주어진 연결 리스트를 병합한 뒤 결과는 다음과 같다:
연결 리스트 1: 1 4 2 5 3 6
연결 리스트 2: 7
```

3. (moveOddItemsToBackLL) 연결 리스트의 모든 홀수를
리스트 뒤쪽으로 이동시키는 C 함수 moveOddItemsToBackLL()을 작성하시오.

함수 원형은 다음과 같다:

```
void moveOddItemsToBackLL(LinkedList *ll);
```

예시 입력/출력 세션은 다음과 같다:
연결 리스트가 2,3,4,7,15,18이라면:
홀수를 연결 리스트 뒤로 이동한 뒤 결과 연결 리스트는
2 4 18 3 7 15 이다.

연결 리스트가 2,7,18,3,4,15라면:
홀수를 연결 리스트 뒤로 이동한 뒤 결과 연결 리스트는
2 18 4 7 3 15 이다.

현재 연결 리스트가 1,3,5라면:
홀수를 연결 리스트 뒤로 이동한 뒤 결과 연결 리스트는
1 3 5 이다.

현재 연결 리스트가 2 4 6이라면:
홀수를 연결 리스트 뒤로 이동한 뒤 결과 연결 리스트는
2 4 6 이다.

4. (moveEvenItemsToBackLL) 연결 리스트의 모든 짝수를

리스트 뒤쪽으로 이동시키는 C 함수 moveEvenItemsToBackLL()을 작성하시오.

함수 원형은 다음과 같다:

```
void moveEvenItemsToBackLL(LinkedList *ll);
```

예시 입력/출력 세션은 다음과 같다:

연결 리스트가 2,3,4,7,15,18이라면:
짝수를 연결 리스트 뒤로 이동한 뒤 결과 연결 리스트는
3 7 15 2 4 18 이다.

연결 리스트가 2,7,18,3,4,15라면:
짝수를 연결 리스트 뒤로 이동한 뒤 결과 연결 리스트는
7 3 15 2 18 4 이다.

현재 연결 리스트가 1,3,5라면:
짝수를 연결 리스트 뒤로 이동한 뒤 결과 연결 리스트는
1 3 5 이다.

현재 연결 리스트가 2 4 6이라면:
짝수를 연결 리스트 뒤로 이동한 뒤 결과 연결 리스트는
2 4 6 이다.

5. (frontBackSplitLL) 단일 연결 리스트를

앞 절반과 뒤 절반의 두 서브리스트로 나누는 C 함수 frontBackSplitLL()을 작성하시오.
원소 수가 홀수이면 남는 하나는 앞 리스트에 들어가야 한다.
frontBackSplitLL()는 frontList와 backList 두 리스트를 출력해야 한다.

함수 원형은 다음과 같다:

```
void frontBackSplitLL(LinkedList *ll, LinkedList
*resultFrontList, LinkedList *resultBackList);
```

예를 들어 주어진 연결 리스트가 2,3,5,6,7이라면,

결과 연결 리스트 frontList와 backList는 다음과 같다:
frontList: 2,3,5
backList: 6,7

예시 입력/출력 세션은 다음과 같다:

```
1: 정수를 연결 리스트에 삽입:
2: 연결 리스트 출력:
3: 연결 리스트를 frontList와 backList 두 개로 분할:
0: 종료:

메뉴를 선택하세요(1/2/3/0): 1
```

```
연결 리스트에 추가할 정수를 입력하세요: 2
결과 연결 리스트는: 2

메뉴를 선택하세요(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하세요: 3
결과 연결 리스트는: 2 3

메뉴를 선택하세요(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하세요: 5
결과 연결 리스트는: 2 3 5

메뉴를 선택하세요(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하세요: 6
결과 연결 리스트는: 2 3 5 6

메뉴를 선택하세요(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하세요: 7
결과 연결 리스트는: 2 3 5 6 7

메뉴를 선택하세요(1/2/3/0): 2
결과 연결 리스트는: 2 3 5 6 7

메뉴를 선택하세요(1/2/3/0): 3
주어진 연결 리스트를 분할한 뒤 결과는 다음과 같다:
앞쪽 연결 리스트: 2 3 5
뒤쪽 연결 리스트: 6 7
```

6. (moveMaxToFront) 연결 리스트를 최대 한 번만 순회한 뒤,

가장 큰 값을 가진 노드를 리스트 맨 앞으로 이동시키는 C 함수 moveMaxToFront()를 작성하시오.
`front->next == max->next; max->next = *ptrHead; *ptrHead = max;`
리스트이다.

함수 원형은 다음과 같다:

```
int moveMaxToFront(ListNode **ptrHead);
```

예를 들어 연결 리스트가 (30, 20, 40, 70, 50)이라면 결과 연결 리스트는
(70, 30, 20, 40, 50)이다.

```
1: 정수를 연결 리스트에 삽입:
2: 가장 큰 값을 가진 노드를 리스트 앞쪽으로 이동:
리스트:
0: 종료:

메뉴를 선택하세요(1/2/0): 1
연결 리스트에 추가할 정수를 입력하세요: 30
연결 리스트는: 30
메뉴를 선택하세요(1/2/0): 1
연결 리스트에 추가할 정수를 입력하세요: 20
연결 리스트는: 30 20
메뉴를 선택하세요(1/2/0): 1
연결 리스트에 추가할 정수를 입력하세요: 40
연결 리스트는: 30 20 40
```

```
메뉴를 선택하세요(1/2/0): 1
연결 리스트에 추가할 정수를 입력하세요: 70
연결 리스트는: 30 20 40 70
메뉴를 선택하세요(1/2/0): 1
연결 리스트에 추가할 정수를 입력하세요: 50
연결 리스트는: 30 20 40 70 50

메뉴를 선택하세요(1/2/0): 2
결과 연결 리스트는: 70 30 20 40 50

메뉴를 선택하세요(1/2/0): 0
```

7. (recursiveReverse) 주어진 연결 리스트의 next 포인터와 head 포인터를 변경하여

재귀적으로 역순으로 만드는 C 함수 recursiveReverse()를 작성하시오.

함수 원형은 다음과 같다:

```
void recursiveReverse(ListNode **ptrHead);
```

예를 들어 연결 리스트가 (1, 2, 3, 4, 5)라면 결과 연결 리스트는 (5, 4, 3, 2, 1)이다.

```
*ptrHead = rest;
first-next = NULL;
first-next-next = first;
```

예시 입력/출력 세션은 다음과 같다:

```
1: 정수를 연결 리스트에 삽입:
2: 연결 리스트 뒤집기:
0: 종료:

메뉴를 선택하세요(1/2/0): 1
연결 리스트에 추가할 정수를 입력하세요: 1
결과 연결 리스트는: 1
메뉴를 선택하세요(1/2/0): 1
연결 리스트에 추가할 정수를 입력하세요: 2
결과 연결 리스트는: 1 2
메뉴를 선택하세요(1/2/0): 1
연결 리스트에 추가할 정수를 입력하세요: 3
결과 연결 리스트는: 1 2 3
메뉴를 선택하세요(1/2/0): 1
연결 리스트에 추가할 정수를 입력하세요: 4
결과 연결 리스트는: 1 2 3 4
메뉴를 선택하세요(1/2/0): 1
연결 리스트에 추가할 정수를 입력하세요: 5
결과 연결 리스트는: 1 2 3 4 5

메뉴를 선택하세요(1/2/0): 2
원소를 뒤집은 뒤 결과 연결 리스트는: 5 4 3 2 1

메뉴를 선택하세요(1/2/0): 0
```